

Computational Thinking

Week-7 Summary

Necessary of Dictionaries in earlier applications(Lecture 7.1):

- Let's you are seen a card and make in separate table or pile
- To avoid other tables/piles, we record that the status of seen as true
- Here the dictionaries are useful

Example1:

Find equal Maths marks pairs

Dictionary: seen
 2 : true
 16 : true

Pairs: Here pairs below are equal marks in maths similar to card no. 16 which is 87
[16, 4] [16, 25] and so on

Example2:

Making grades to each student/card no.

Dictionary: grade
 {12 : B, 23 : C, 14 : A....}

Example3:

Words with same letter count

Dictionary: lettercount
 {
 2: 9,
 3: 15,
 6: 7,
 }

Note: When new letter count is found set it's initial value to 1. Then increment by one after each finding of similar letter count word. Here indirectly compute something from existing data.

Example4:

Score dataset "city count" with similar city/town

city count
{
 Chennai: 8
 Banglore: 5
 Madhurai: 5
 ...}

Dictionaries to solve birthday paradox(Lecture 7.2):

- Earlier we used nested iterations
- Bining within the same month DOBs
- Sorting by DOB is also another approach

Using dictionaries it is very easier i.e., "day month" as key and sequence number as value

{
 "15 Mar" : 21, 7
 "23 Mar" : 6
 }

Pseudocode for dictionaries(Lecture 7.3):

```
chemMarks = {}  
while (Table 1 has more rows) {  
    Read the first row X in Table 1  
    name = X.Name  
    marks = X.ChemistryMarks  
    chemMarks[name] = marks  
}  
Move X to Table 2  
}
```

Processing the dictionaries:

Iterating through dictionary “chemMarks” and compute avg marks in chemistry

```
total = 0  
count = 0  
foreach k in keys(chemMarks) {  
    total = total + chemMarks[k]  
    count = count + 1  
}  
chemavg = total/count
```

Relations among customers based on their spending patterns part 1(Lecture 7.4):

Customer name as key and category of purchases as values from shopping bills dataset.

	A	F	T	U	H
Abhinav :	34	8	2		1
Neeraja :					
Akshaya :					

Based on the total of all purchases from each customer. We can find similar customers who are buying more than some X items in food or any category.

Relations among customers based on their spending patterns part-2 (Lecture 7.5):

Compare the each pair of customers and compute the distance between each category of purchases. Here use absolute value.

1, 2 - 7	2, 3 - 17
1, 3 - 20	2, 4 - 12
....	...
...	

Introduction to dictionaries(Lecture 7.6):

- Build a dictionary of store name as key and count of food items as value

Food
{ SV Stores : 73,
Sun General : 9,
Big Bazar : 10 }

Based on the above dictionary, we come to conclusion that SV Stores famous for "Food"

- Build another dictionary of customer name as key and count of food items as value

Customer Food
{ Rajesh : 2,
Abhinav : 0,
Akshaya : 9
.... }

Pseudocode for real time examples using Dictionaries:

Example 1:

Customers buying food items:

```
foodD = {}  
while (Table 1 has more rows) {  
    Read the first row X in Table 1  
    customer = X.CustomerName  
    items = X.Items  
    foreach row in items {  
        if (row.Category == "Food") {  
            if (isKey(foodD, customer)) {  
                foodD[customer] = foodD[customer] + 1  
            }  
            else {  
                foodD[customer] = 1  
            }  
        }  
    }  
    Move X to Table 2  
}
```

Example 2:

Birthday paradox:

Store the students associated with each date of birth as a dictionary

```

birthdays = {}
duplicates = {}
while (Table 1 has more rows) {
    Read the first row X in Table 1
    dob = X.DoB
    seqno = X.SeqNo
    if (isKey(birthdays,dob)) {
        duplicates[dob] = True
        birthdays[dob][seqno] = True
    }
    else {
        birthdays[dob] = {}
        birthdays[dob][seqno] = True
    }
    Move X to Table 2
}

```

Example 3(Nearest noun preceding the pronoun):

Pseudocode to create a dictionary with part of speech as keys, sorted list of card numbers as values:

```

partOfSpeech = {}
partOfSpeech['Noun'] = []
partOfSpeech['Pronoun'] = []
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (X.PartOfSpeech == 'Noun') {
        partOfSpeech['Noun'] = partOfSpeech['Noun'] ++ [X.SerialNo]
    }
    if (X.PartOfSpeech == 'Pronoun') {
        partOfSpeech['Pronoun'] = partOfSpeech['Pronoun'] ++
            [X.SerialNo]
    }
}

```

Move X to Table 2

}

Pseudocode to iterate through the dictionary to match pronouns:

```
matchD = {}
foreach p in partOfSpeech['Pronoun'] {
    matched = -1
    foreach n in partOfSpeech['Noun'] {
        if (n < p) {
            matched = n
        }
        else {
            exitloop
        }
    }
    matchD[p] = matched
}
```

Note: that `partOfSpeech['Noun']` and `partOfSpeech['Pronoun']` are both sorted in ascending order of `SerialNo`.